

VISUALIZATION

Visualization is a way of representing data in graphical or pictorial format for better understanding of data.

Data visualization helps identify patterns, trends, and relationships in data easily.

Python provides multiple visualization libraries such as Matplotlib, Seaborn, Plotly, Bokeh, and Pandas Visualization.

In this documentation, we will study Pandas Visualization and Matplotlib.

MATPLOTLIB

Matplotlib is one of the most popular Python libraries for data visualization.

It was introduced by John Hunter in 2002 and is built on NumPy.

It provides complete control over graphs and supports multiple chart types.

Components of Matplotlib :

1. Figure

Figure is the main container in which all plots are present. A figure can contain a single plot or multiple plots.

2. Axes

Axes are the actual plots attached to the figure. They contain the data visualization area. Functions like `set_xlabel()` and `set_ylabel()` are used to set labels for x-axis and y-axis.

3. Axis

Axis is responsible for generating scales, ticks, and limits on the graph. It helps in representing data values properly on x and y directions.

4. Artists

Everything visible inside a figure is called an artist. This includes text, lines, labels, titles, legends, and plots.

Pyplot in Matplotlib:

Pyplot is a submodule of Matplotlib that contains functions used for creating graphs and charts easily. It provides a simple interface similar to MATLAB for plotting data.

Pyplot is mainly used for:

- Line charts
- Bar graphs
- Scatter plots
- Histograms
- Pie charts
- Area charts

Importing Libraries:

```
import matplotlib.pyplot as plt
```

Importing numpy:

```
import numpy as np
```

1. LINE CHART

A line chart is used to represent data points connected by straight lines. It is useful for showing trends, changes, and comparisons over time. The graph makes it easy to understand increases or decreases in values clearly. In Matplotlib, the plot() function is used to create a line chart.

Use Case: Sales trends, temperature changes, performance analysis.

Code Snippet:

```
import matplotlib.pyplot as plt

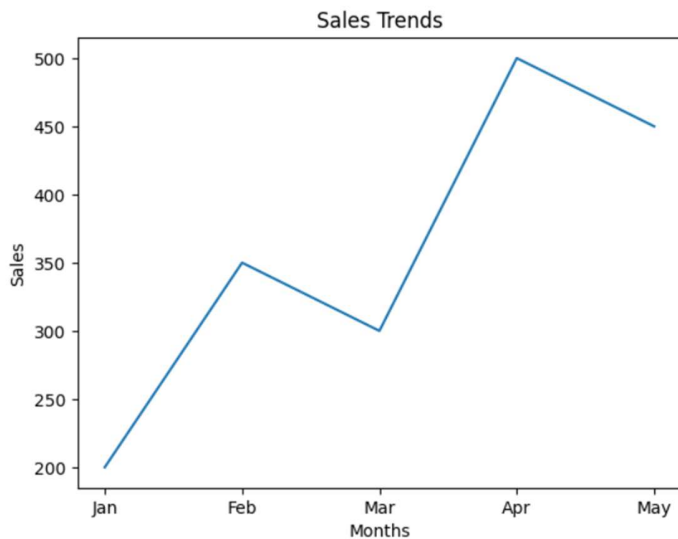
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [200, 350, 300, 500, 450]

plt.plot(months, sales)

plt.title("Sales Trends")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.show()
```

Output:



Description:

This code uses Pyplot to create a line graph for monthly sales data.

The months list represents the x-axis and the sales list represents the y-axis values.

plot() is used to draw the graph, while title(), xlabel(), and ylabel() add labels and title to the chart.

Finally, show() displays the graph on the screen.

2. BAR GRAPH

A bar graph is used to compare different categories using rectangular bars. The length or height of each bar represents the value of that category. It helps in understanding and comparing data easily in a visual form. In Matplotlib, the bar() function is used to create bar graphs.

Code Snippet:

```
import matplotlib.pyplot as plt
import numpy as np

subjects = ["Math", "Science", "English", "Hindi", "Computer"]
marks = np.random.randint(60, 100, 5)

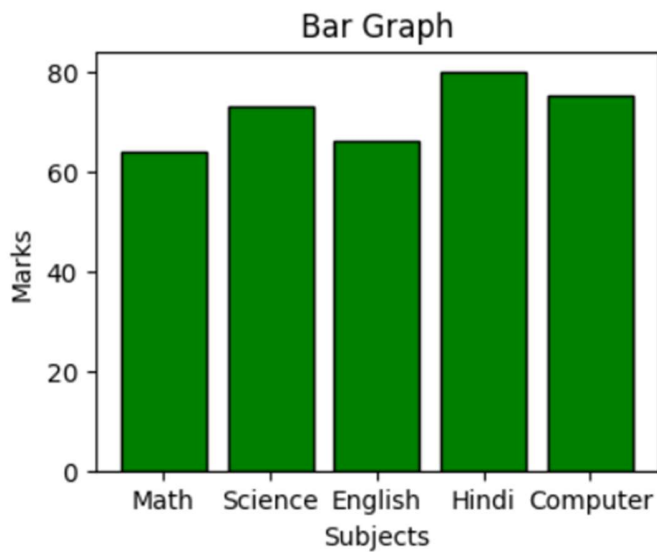
fig = plt.figure(figsize=(4, 3))

plt.bar(subjects, marks, color="green", edgecolor="black")

plt.title("Bar Graph")
plt.xlabel("Subjects")
plt.ylabel("Marks")

plt.show()
```

Output:



Description:

`bar()` is used to create the bar graph.

`subjects` contains the names of the subjects shown on the x-axis.

`marks` contains the values represented by the height of each bar.

`color='green'` changes the color of the bars.

`edgecolor='black'` adds black borders to the bars.

`title()` is used to add the title of the graph.

`xlabel()` labels the x-axis.

`ylabel()` labels the y-axis.

`show()` displays the graph on the screen.

3. HISTOGRAM

A histogram is used to represent the distribution of numerical data in graphical form. The data is divided into intervals called bins, and the height of each bar shows the frequency of values in that interval. It helps in understanding the spread and pattern of the data clearly.

Code Snippet:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(100, 50, 100)

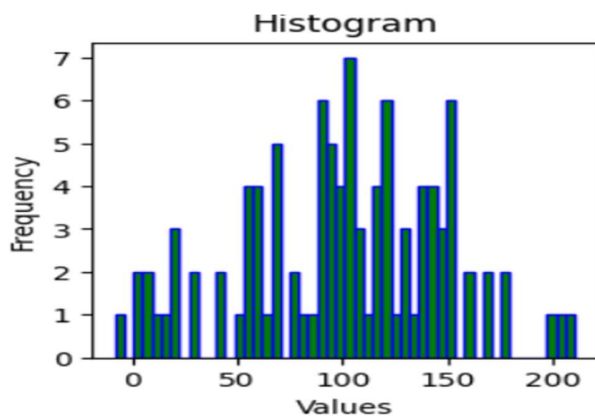
fig = plt.figure(figsize=(3, 3))

plt.hist(x, bins=50, color="green", edgecolor="blue")

plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")

plt.show()
```

Output:



Description:

hist() is used to create the histogram.

x contains the numerical data values used in the graph.

bins=50 divides the data into 50 intervals or groups.

color='red' changes the color of the histogram bars.

edgecolor='blue' adds blue borders to the bars.

title() is used to add the title of the graph.

ylabel() labels the y-axis.

xlabel() labels the x-axis.

show() displays the histogram on the screen.

4. SCATTER PLOT

A scatter plot displays data points on a graph to show the relationship between two variables. Each point represents a pair of values on the x-axis and y-axis. It helps in identifying patterns and trends in the data.

Code Snippet:

```
import matplotlib.pyplot as plt
import numpy as np

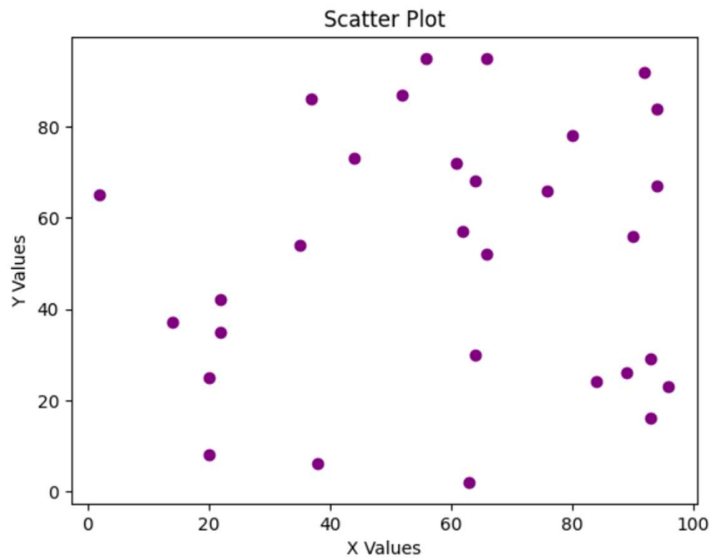
x = np.random.randint(1, 100, 30)
y = np.random.randint(1, 100, 30)

plt.scatter(x, y, color='purple')

plt.title("Scatter Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")

plt.show()
```

Output:



Description:

`scatter()` is used to create the scatter plot.

`x` and `y` contain the data points plotted on the graph.

`color='purple'` changes the color of the points.

`title()` adds the graph title.

`xlabel()` and `ylabel()` label the x-axis and y-axis.

`show()` displays the graph on the screen.

5. PIE CHART

A pie chart represents data in the form of slices of a circle. Each slice shows the proportion or percentage of a category in the complete dataset. It helps in understanding the contribution of different categories easily. Different colors are used to make the chart more attractive and readable.

Code Snippet:

```
import matplotlib.pyplot as plt

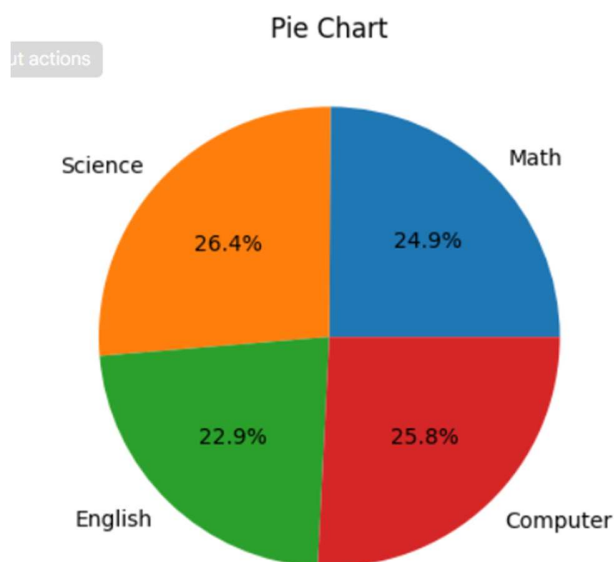
subjects = ["Math", "Science", "English", "Computer"]
marks = [85, 90, 78, 88]

plt.pie(marks, labels=subjects, autopct='%1.1f%%')

plt.title("Pie Chart")

plt.show()
```

Output:



Description:

`pie()` is used to create the pie chart.

`marks` contains the values represented in the chart.

`labels=subjects` adds labels to each slice.

`autopct='%1.1f%%'` displays percentage values on the chart.

`title()` adds the title of the chart.

`show()` displays the chart on the screen.

PANDAS VISUALIZATION

Pandas is mainly used for data analysis, but it also provides visualization features using Matplotlib internally.

Pandas plotting functions are simple and beginner-friendly.

Components of pandas:

Series – A one-dimensional labeled array used to store single-column data.

DataFrame – A two-dimensional table-like structure with rows and columns. It is the most commonly used Pandas component.

Panel – A three-dimensional data structure used for handling large datasets (now less commonly used in modern Pandas).

Importing Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

1. LINE PLOT

A line plot in Pandas is used to display data points connected by straight lines. It is useful for showing trends and changes over time. Pandas uses Matplotlib functions internally to create line plots easily from datasets. The graph helps in understanding the increase or decrease in values clearly.

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Months": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [200, 350, 300, 500, 450]
}

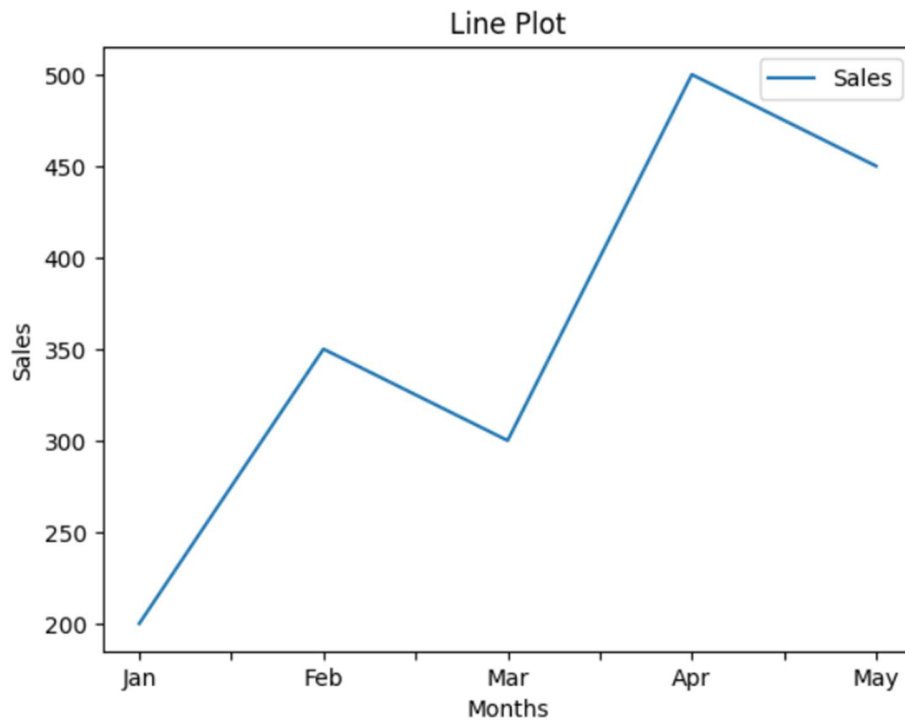
df = pd.DataFrame(data)

df.plot(x="Months", y="Sales", kind="line")

plt.title("Line Plot")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.show()
```

Output:



Description:

DataFrame() is used to create the dataset in table form.

plot() is used to create the line plot.

x="Months" sets the x-axis values.

y="Sales" sets the y-axis values.

kind="line" specifies the line graph.

title() adds the graph title.

xlabel() and ylabel() label the axes.

show() displays the graph on the screen.

2. BAR PLOT

A bar plot in Pandas is used to compare values of different categories using rectangular bars. It helps in visualizing differences between data clearly and easily. Pandas provides simple plotting functions to create bar graphs directly from datasets.

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt

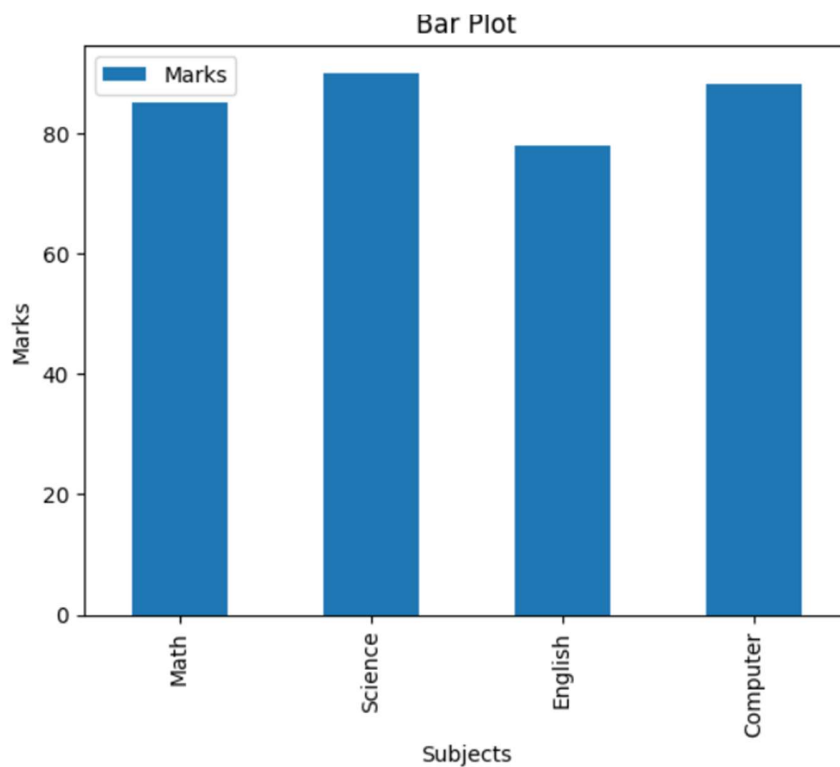
data = {
    "Subjects": ["Math", "Science", "English", "Computer"],
    "Marks": [85, 90, 78, 88]
}
df = pd.DataFrame(data)

df.plot(x="Subjects", y="Marks", kind="bar")

plt.title("Bar Plot")
plt.xlabel("Subjects")
plt.ylabel("Marks")

plt.show()
```

Output:



Description:

`DataFrame()` is used to create the dataset.

`plot()` is used to create the graph.

`kind="bar"` specifies the bar plot.

`x="Subjects"` sets the x-axis values.

`y="Marks"` sets the y-axis values.

`title()` adds the graph title.

`xlabel()` and `ylabel()` label the axes.

`show()` displays the graph on the screen.

3. HISTOGRAM

A histogram in Pandas is used to represent the distribution of numerical data. It groups data into intervals called bins and shows the frequency of values in each interval. It helps in understanding the spread and pattern of the dataset clearly.

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
}

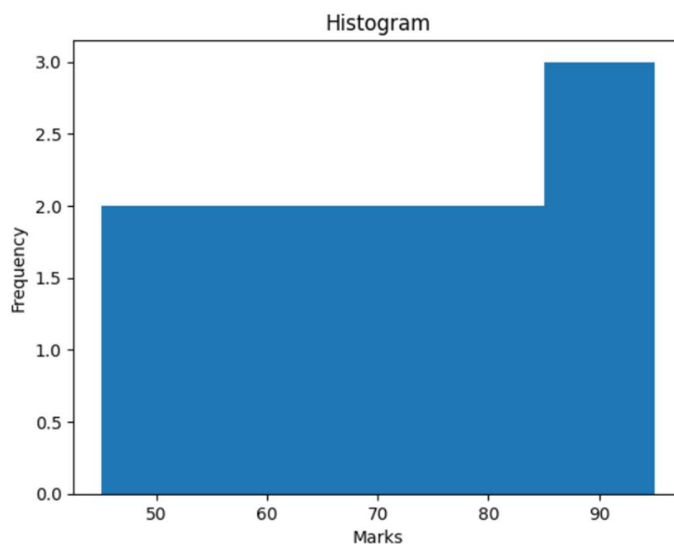
df = pd.DataFrame(data)

df["Marks"].plot(kind="hist", bins=5)

plt.title("Histogram")
plt.xlabel("Marks")
plt.ylabel("Frequency")

plt.show()
```

Output:



Description:

DataFrame() is used to create the dataset.

plot() is used to create the graph.

kind="hist" specifies the histogram.

bins=5 divides the data into 5 intervals.

title() adds the graph title.

xlabel() and ylabel() label the axes.

show() displays the histogram on the screen.

4. BOX PLOT

A box plot in Pandas is used to represent the distribution of numerical data. It shows the minimum value, maximum value, median, and quartiles of the dataset. It also helps in identifying outliers and understanding the spread of data clearly.

Code Snippet:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
}

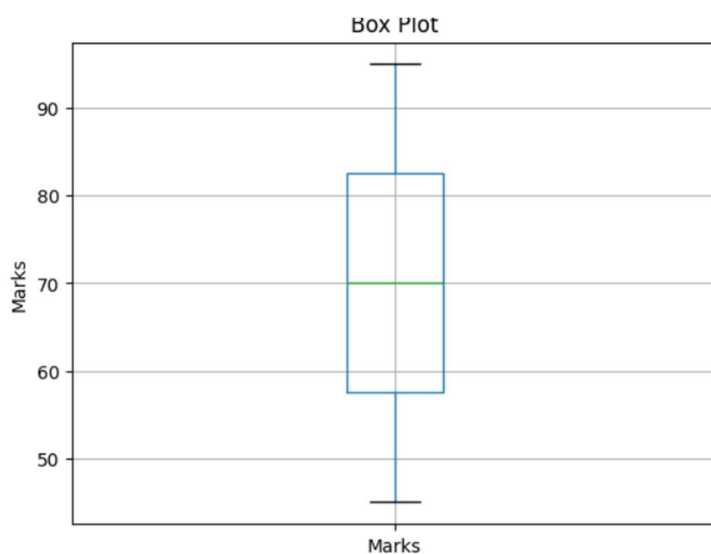
df = pd.DataFrame(data)

df.boxplot(column="Marks")

plt.title("Box Plot")
plt.ylabel("Marks")

plt.show()
```

Output:



Description:

DataFrame() is used to create the dataset.

boxplot() is used to create the box plot.

column="Marks" selects the column for plotting.

title() adds the graph title.

ylabel() labels the y-axis.

show() displays the graph on the screen.

5. PIE CHART

A pie chart in Pandas is used to represent data in the form of slices of a circle. Each slice shows the proportion or percentage of a category in the dataset. It helps in understanding the contribution of different categories clearly and visually.

Code Snippet :

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Subjects": ["Math", "Science", "English", "Computer"],
    "Marks": [85, 90, 78, 88]
}

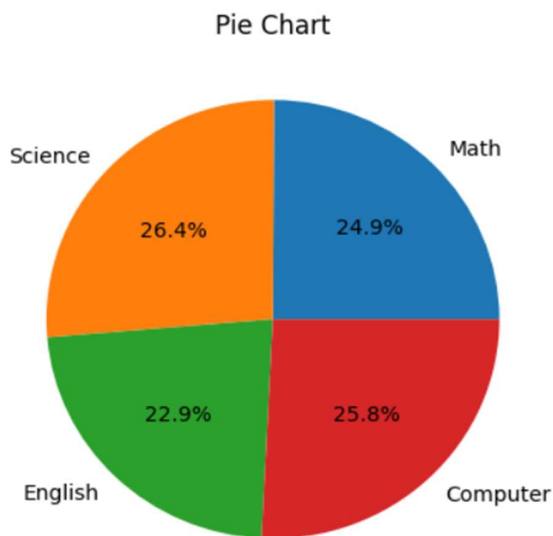
df = pd.DataFrame(data)

df.set_index("Subjects")["Marks"].plot(kind="pie", autopct='%1.1f%%')

plt.title("Pie Chart")
plt.ylabel("")

plt.show()
```

Output :



Description :

`DataFrame()` is used to create the dataset.

`set_index()` sets the subject names as labels.

`plot()` is used to create the graph.

`kind="pie"` specifies the pie chart.

`autopct='%1.1f%%'` displays percentage values on the chart.

`title()` adds the chart title.

`show()` displays the chart on the screen.

6.AREA PLOT

An area plot is used to represent changes in data over time using filled areas below the lines. It helps in understanding trends and comparing quantities visually. Different colors make the graph more attractive and easy to read.

Code Snippet :


```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Months": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [200, 350, 300, 500, 450]
}

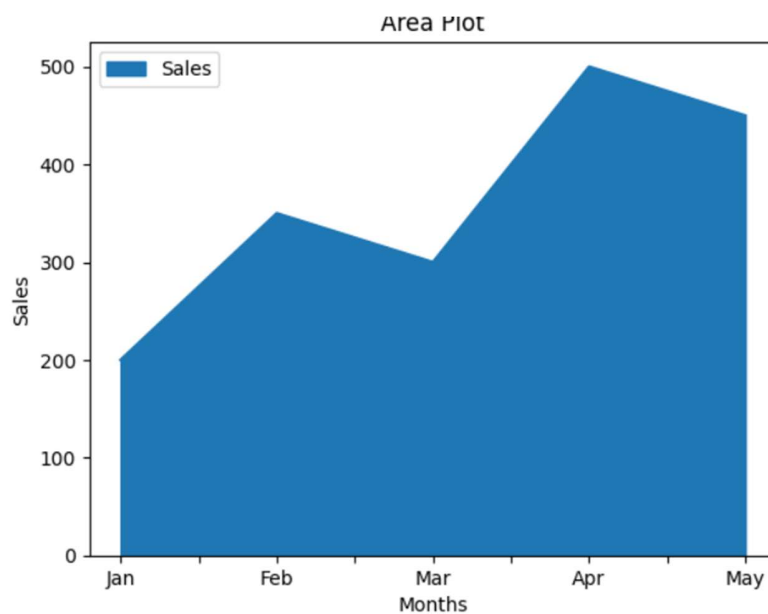
df = pd.DataFrame(data)

df.plot(x="Months", y="Sales", kind="area")

plt.title("Area Plot")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.show()
```

Output:



Description :

DataFrame() is used to create the dataset.

plot() is used to create the graph.

kind="area" specifies the area plot.

x="Months" sets the x-axis values.

y="Sales" sets the y-axis values.

title() adds the graph title.

xlabel() and ylabel() label the axes.

show() displays the graph on the screen.

7. DENSITY PLOT

A density plot is used to show the distribution of continuous numerical data in a smooth curve form. It helps in understanding the concentration and spread of data values clearly. The graph provides a smoother representation compared to a histogram

Code Snippet :

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Marks": [45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
}

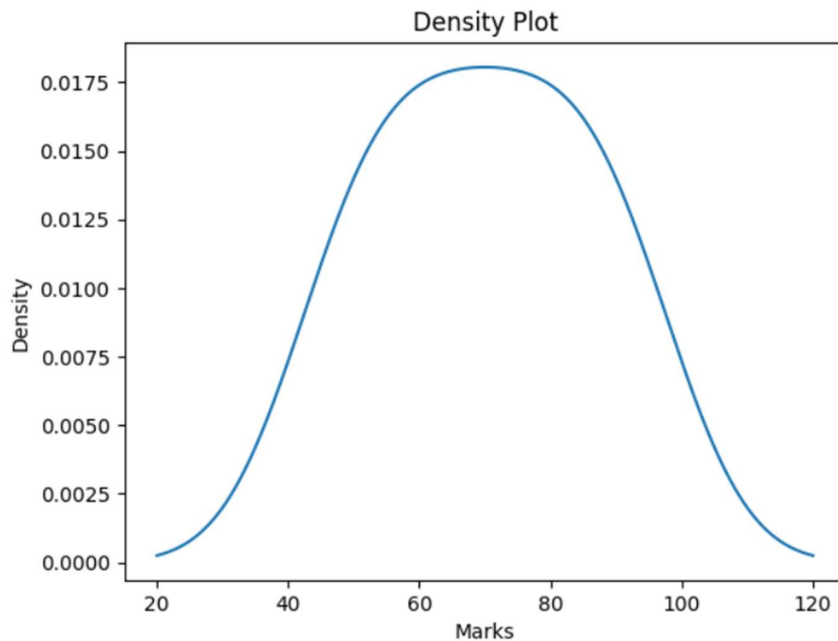
df = pd.DataFrame(data)

df["Marks"].plot(kind="density")

plt.title("Density Plot")
plt.xlabel("Marks")

plt.show()
```

Output :



Description :

`DataFrame()` is used to create the dataset.

`plot()` is used to create the graph.

`kind="density"` specifies the density plot.

`title()` adds the graph title.

`xlabel()` labels the x-axis.

`show()` displays the graph on the screen.

8. HEXBIN PLOT

A hexbin plot is used to represent the relationship between two numerical variables using hexagonal bins. It is useful for visualizing large datasets where scatter plots may become crowded. Different color intensities represent the density of data points in each hexagon.

Code Snippet :

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

data = {
    "x": np.random.randn(1000),
    "y": np.random.randn(1000)
}

df = pd.DataFrame(data)

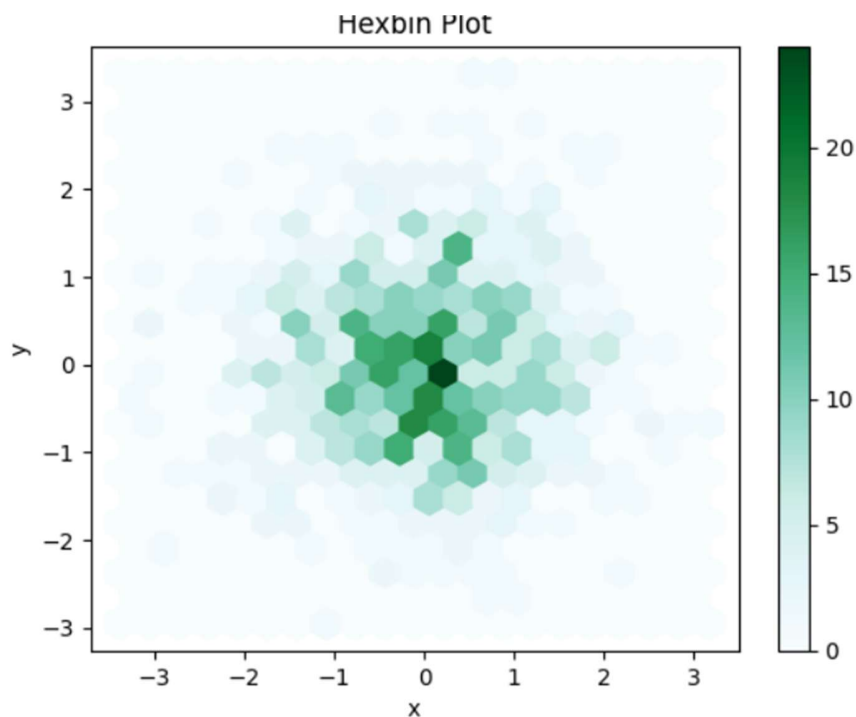
df.plot(kind="hexbin", x="x", y="y", gridsize=20)

plt.title("Hexbin Plot")

plt.show()

```

Output :



Description :

DataFrame() is used to create the dataset.

plot() is used to create the graph.

`kind="hexbin"` specifies the hexbin plot.

`x` and `y` define the axes values.

`gridsize=20` sets the number of hexagons in the graph.

`title()` adds the graph title.

`show()` displays the graph on the screen.

COMPARISON OF MATPLOTLIB AND PANDAS VISUALIZATION

Matplotlib:

- **Visualization Library:** Matplotlib is a powerful Python library used for creating static, animated, and interactive visualizations. It provides detailed control over graphs and charts.
- **Flexibility:** Matplotlib supports many types of plots such as line plots, bar graphs, histograms, scatter plots, pie charts, and more. Users can fully customize the appearance of plots.
- **Customization:** It allows customization of colors, labels, markers, line styles, titles, axes, and figure sizes, making it suitable for professional and publication-quality graphs.
- **Low-Level Control:** Matplotlib gives fine-grained control over every part of the graph, which is useful for advanced visualizations.
- **Integration:** It works well with libraries like NumPy and Pandas for data analysis and visualization.

Advantages of Matplotlib:

- High level of customization and flexibility.
 - Supports many different plot types.
 - Suitable for detailed and advanced visualizations.
 - Strong community support and documentation.
 - Useful for creating high-quality graphs and charts.
-

Pandas Visualization:

- **Built-in Plotting:** Pandas provides built-in visualization features that use Matplotlib internally for plotting data directly from DataFrames.
- **Simple Syntax:** Pandas makes plotting easy with simple commands like `plot()`, reducing the amount of code required.
- **Data Analysis Integration:** Since Pandas is mainly used for data handling and analysis, visualization can be performed directly on datasets without extra processing.
- **Common Plot Types:** Pandas supports line plots, bar plots, histograms, scatter plots, box plots, area plots, and more.
- **Quick Visualization:** It is useful for quick and simple data visualization during exploratory data analysis.

Advantages of Pandas Visualization:

- Easy and simple syntax for plotting.
- Direct plotting from DataFrames and Series.
- Less code compared to Matplotlib.
- Good for quick data analysis and visualization.
- Seamless integration with data manipulation features.

Overall, the choice between Matplotlib and Pandas depends on the requirements of the data visualization task. Matplotlib is preferred for its flexibility, advanced customization, and detailed control over plots, while Pandas is preferred for its simple syntax and quick visualization directly from datasets. Matplotlib is suitable for creating complex and professional graphs, whereas Pandas is ideal for fast and easy data analysis and basic plotting. Many users use both libraries together, combining the data handling capabilities of Pandas with the visualization power of Matplotlib.

